

A Python-DPCTL Path for miniLB Lattice Boltzmann Simulations on SYCL Backends

Roman Andrei, Bianca-Maria Andreica, Ariana Crista
Department of Computer Science
University of Salerno
Italy

Abstract—In this project we start with the original miniLB Lattice Boltzmann code and extend it with the possibility to run and benchmark it from Python. We don't replace the solver itself with a Python implementation, as the numerical calculations have to remain in compiled C++/SYCL kernels. Python is used for device selection, allocation of the simulation arrays, call of the native extension, validation of the run, and benchmarking.

The final version uses pybind11, dpctl, and USM shared memory to interconnect Python with the C++/SYCL implementation. We compare the original miniLB SYCL CPU/OpenCL baseline with the Python-DPCTL version and run the final version on NVIDIA Tesla V100S GPU via the CUDA SYCL backend. Results illustrate the overhead of the Python facing interface on CPU/OpenCL and the scalability of the final V5 version on GPU grids.

I. INTRODUCTION

Lattice Boltzmann Methods (LBM) are widely used in fluid simulations as they represent the evolution of the fluid as local calculations over a regular lattice. It is an appealing approach for high performance computing since each lattice cell performs similar calculations, and several cells can perform computations in parallel. At the same time, a good implementation should address several issues related to memory layout, boundary conditions, selection of the device, and benchmarking. It is especially important in the case when the same numerical code should be executed on several backends, since changing the execution device should not change the mathematical update performed by the solver.

Motivation. The beginning of this work is in the original miniLB code, which provides a C++/SYCL implementation of a Lattice Boltzmann solver. We do not intend to create an entirely new simulator in Python. Instead, this project investigates how the high performance path can be exposed to Python while preserving the compiled C++/SYCL execution of the computationally expensive operations. This is interesting, since Python is suitable for experimentation, parameter selection, device selection, and post-processing, while C++/SYCL is more adequate for the computational kernels. In practice, this separation corresponds to how scientific workflows usually look like: researchers prefer a Python interface for quick testing, but they still need to perform the actual numerical kernel on the most proper hardware backend.

The main idea is to utilize Python as a control layer above the native implementation. Benchmarking script creates the SYCL queue, allocates the memory, selects the target device, calls

the extension module, and checks the result. The LBM step and the whole benchmarking loop are performed in compiled C++/SYCL code. This separation was used throughout the project to avoid a typical error in Python bindings for numerical kernels: making the Python layer responsible for fine-grained operations that should be performed inside the native code.

Contribution. The final version of the project, called V5 in our development process, combines pybind11 bindings, dpctl queues, Python allocated USM shared memory, C++/SYCL kernels, dynamic device selection, correctness check, and benchmarking logging. The implementation was tested on CPU/OpenCL backend and on NVIDIA Tesla V100S GPU using the CUDA SYCL backend. The CPU/OpenCL experiments compare the original miniLB SYCL version against the Python-DPCTL version, while GPU experiments show how the same design can be run on real CUDA-capable accelerator.

The report is focused on correctness, interoperability, and performance characteristics rather than on claims that the Python-facing version is always faster than the original specialized baseline.

Related work. LBM is a popular computational method for fluid dynamics simulations, which is usually mapped to parallel hardware due to its local stencil-like structure [1]. SYCL provides a single source C++ programming model for heterogeneous devices [2]. dpctl allows to access SYCL concepts, like devices, queues, and USM allocations, from Python [3]. pybind11 is used to expose the C++ code to Python with the minimal overhead in comparison with re-implementing the computation in Python [4].

In this sense, this project follows the same path: Python provides the higher level interface, while the performance critical execution remains in C++/SYCL. As compared to the pure Python implementation, this design tries to avoid moving the main numerical work away from the optimized native backend. As compared to the usage of the original command line executable, it provides more control for the Python side for experimentation and benchmarking automation.

II. BACKGROUND

This subsection briefly introduces basic notions needed for the understanding of the implementation: D2Q9 Lattice Boltzmann model, SYCL execution, USM memory and the Python binding path.

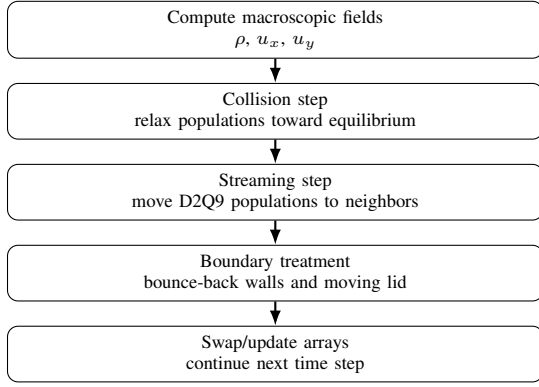


Fig. 1. Logical structure of one D2Q9 LBM time step.

D2Q9 Lattice Boltzmann model. Current implementation uses two-dimensional D2Q9 LBM model. Each cell stores nine distribution values for a rest direction, and four diagonal directions. On each time step, the algorithm calculates macroscopic values (density and velocity), does collision update, streams distributions to neighbor cells and applies boundary conditions.

The project uses the Lid-Driven Cavity use case. The walls use bounce-back boundary conditions, and the top lid has a prescribed horizontal velocity. This case is interesting because it requires both regular LBM updates and boundary conditions, which means that it tests not only periodic streaming pattern.

In D2Q9 model, collision stage relaxes distribution values to the equilibrium distribution based on density and velocity. Streaming stage streams each population in its corresponding lattice direction. In simplified notation, update can be considered as

$$f_i^*(\mathbf{x}, t) = f_i(\mathbf{x}, t) + \omega (f_i^{eq}(\rho, \mathbf{u}) - f_i(\mathbf{x}, t)),$$

followed by streaming of each direction i to its neighbor cell. From the implementation point of view, it means that each time step accesses all cells of the lattice and all nine populations. Thus, memory access patterns and optimization of unnecessary memory transfers play important role in the implementation performance.

Figure 1 shows the logical sequence of one LBM time step implemented in the project. This diagram is deliberately high-level, because the aim is to depict operations which have to be performed inside compiled kernel path, not all array indexes used in the code.

SYCL and device execution. SYCL is used to express the parallel kernels operating on the lattice. A SYCL queue represents the selected device and is used to submit work. The same implementation idea is used in this project for CPU/OpenCL and CUDA GPU execution, however the code should be compiled with appropriate back-end target. For NVIDIA Tesla V100S GPU, extension was rebuilt for CUDA SYCL target with `nvptx64-nvidia-cuda` target and the `sm_70` architecture flag. This is important, because SYCL expresses the parallel program in a portable way, however the

TABLE I
SEPARATION OF RESPONSIBILITIES IN THE FINAL IMPLEMENTATION.

Python / dpctl side	C++/SYCL side
Device and queue selection	LBM collision kernel
USM memory allocation	Streaming and boundary update
Benchmark configuration	5000-step time loop
Validation collection	Timing and native execution
CSV/log management	Direct access to USM pointers

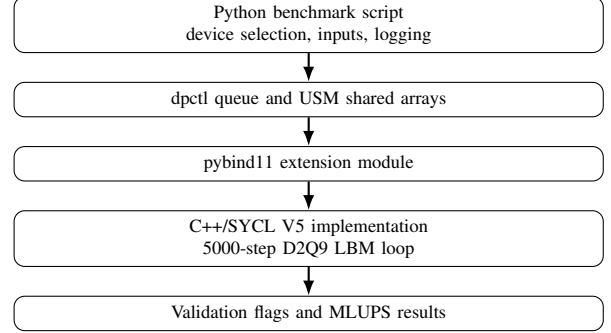


Fig. 2. Execution path of the V5 Python-DPCTL interface.

generated code should correspond to the device backend and architecture used for execution.

DPCTL and USM. dpctl is the Python layer for creation of SYCL queues and USM memory allocation. USM is important here, because it makes it possible to pass memory allocated from Python to the C++/SYCL side without copying arrays into the additional Python-only representation. Arrays used in this implementation are the LBM distributions and the macroscopic density and velocity arrays

III. IMPLEMENTATION OF LATTICE BOLTZMANN WITH DPCTL

This subsection describes how the path with Python/DPCTL interface was added around the original C++/SYCL LBM implementation. The key design decision is keeping Python out of the numerical inner loop.

Overall architecture. Figure 2 summarizes the execution path implemented in V5. Python retains responsibility for the high-level benchmark workflow, while repeated LBM computation is done in the native C++/SYCL part.

Python creates the queue, allocates the arrays and invokes one function from the C++ side. C++ side receives the Python objects, extracts the SYCL queue and USM pointers and launches the kernels.

Memory interface. The simulation arrays are allocated in Python using dpctl USM shared memory. These arrays are passed to the native extension through the `__sycl_usm_array_interface__`. This interface is the bridge that lets the C++ side access memory created from Python. It also keeps the benchmark closer to a realistic Python workflow, where the user controls data allocation and device selection from Python.

TABLE II
MAIN ARRAYS USED BY THE PYTHON-DPCTL LBM PATH.

Array group	Role in the simulation
Distribution arrays	Store the nine D2Q9 populations per cell
Stream buffer	Holds streamed values between time steps
Density array	Stores the macroscopic density ρ
Velocity arrays	Store the velocity components u_x and u_y
Validation data	Records checks and final state information

The pybind11 module exposes a compact interface to Python instead of exposing many low-level kernel functions separately. This is critical for the benchmark, as Python should not be responsible for managing individual update steps of the lattice, just launching a calculation and receiving its result at the end.

Kernel organization. The implementation uses arrays of distribution functions before and after collisions/streaming and arrays of density and velocity. C++/SYCL part calculates the D2Q9 collision, streams and applies boundary conditions. The validation flags check that all populations are included, no boundary conditions are substituted with periodic only one, and kernels are launched from C++.

Data flow. On benchmark launch, Python allocates the USM arrays and gives them to the extension module. C++/SYCL works with those allocations directly. On each time step, C++ updates the distribution function arrays using additional temporary storage for streamed values. After the end of the benchmark run, Python receives the timing and validation flags. Such data transfer provides convenient host-side interface, but excludes repetitive transfers between Python-owned arrays and C++-owned arrays.

Benchmark loop. One of the possible designs could include calling a single LBM step from Python 5000 times. It was not used, as it would add additional Python-to-C++ overhead to every time step. In V5, Python calls the extension only once for benchmark run, and 5000-step loop is executed inside C++. It ensures the control interface to stay in Python while placing the repetitive code into compiled implementation.

Arrays are also reused inside the time loop instead of allocation at each time step. Thus, the benchmark measures mainly the LBM update and the overhead of the backend choice, not the repetitive host-side setup. This is critical for the fair comparison, as Python layer should be responsible for configuration and experiment starting, but not for the iteration itself.

Execution sequence. For both the CPU and GPU executions, the benchmark goes through this sequence:

- 1) Python constructs the chosen SYCL queue using dpctl.
- 2) Python allocates the USM arrays needed for the LBM fields
- 3) The pybind11 extension gets the queue and memory objects.
- 4) C++/SYCL executes the 5000-step LBM loop.
- 5) Validation of the result state and writing the timing to the benchmark output.

TABLE III
IMPORTANT BUILD AND RUNTIME CHOICES.

Choice	Reason
Release compilation	Enables compiler optimizations
C++ loop over steps	No need to call Python code repeatedly
Runtime selector	Switches backend without changing Python code
sm_70	Corresponds to the architecture of the Tesla V100S GPU
CSV/log output	Keeps benchmark results reproducible

This sequence keeps the Python part visible and reproducible while keeping the repeated numerical work in the compiled implementation.

Simplified native benchmark loop. The C++ side follows the simplified structure below. The actual implementation contains the SYCL kernel submissions and the details of the D2Q9 arrays, but the important point is that the loop over time steps is not controlled from Python.

```
for step in range(5000):
    compute_rho_and_velocity()
    collide_all_D2Q9_populations()
    stream_all_D2Q9_populations()
    apply_bounce_back_walls()
    apply_moving_lid()
    check_final_state()
    report_elapsed_time_and_MLUPS()
```

Why the time loop is native. One of the critical implementation choices is how the time loop is implemented. If Python calls each C++/SYCL step separately, the benchmark will include 5000 transitions between Python and native extension. Such approach will introduce additional overhead and hide the actual behavior of LBM kernels. For this reason, the exported function starts the whole run, not only single update. Python controls the experiment, but the loop with repetitive update stays in the C++/SYCL implementation.

Design trade-offs. The design includes some trade-offs. Firstly, it includes the layer visible from Python, so it is not expected to behave like a standalone executable in all cases. Nevertheless, the loop with expensive operations is placed inside C++/SYCL, so the overhead introduced by Python is reduced. Therefore, the design attempts to balance the goals of preserving the convenient Python workflow and placing the actual LBM computation into the compiled heterogeneous programming path.

Another important trade-off is related to memory ownership. As Python allocates USM arrays, it is possible to run the benchmark from Python without creating the extra copy of the same data on the host side. However, C++/SYCL should correctly interpret the memory interface and use the right queue context. This is why the implementation includes explicit checks of the Python-side dpctl allocation and C++ kernel execution before reporting the performance number.

Build configuration. The CPU/OpenCL version was built using the Intel DPC++/SYCL compiler in release mode. For the NVIDIA GPU experiment, the native extension was rebuilt for the CUDA SYCL backend using the nvptx64-nvidia-cuda target and the sm_70 architecture

TABLE IV
SUMMARY OF THE MAIN DIFFERENCES BETWEEN THE ORIGINAL BASELINE AND V5.

Feature	Original	V5
Main interface	Executable	Python-callable module
Control layer	C++	Python + dpctl
Memory allocation	Native path	Python USM arrays
Benchmark loop	Native code	Native C++/SYCL loop
CPU/OpenCL run	Yes	Yes
CUDA GPU run	Failed in test	Completed
Validation output	Limited	Explicit flags

flag, which corresponds to the Tesla V100S generation used in the CINECA environment.

Runtime device selection. The device selector is loaded at runtime, hence the same Python benchmark can choose either CPU/OpenCL or CUDA GPU execution path. This became possible due to the capability to perform local CPU/OpenCL test and then the same high-level benchmark on the CINECA Tesla V100S GPU after rebuilding the extension with CUDA SYCL target.

IV. EXPERIMENTAL EVALUATION

This section evaluates the implementation correctness and performance. The main comparison is between the original miniLB SYCL CPU/OpenCL baseline and the V5 Python-DPCTL-SYCL implementation. The GPU performance measurements are also provided for the final V5 implementation.

The experiments will revolve around three practical questions. First, we need to check that Python-DPCTL path does not change the expected behavior and boundary handling of LBM. Second, we measure how big performance penalty is introduced on CPU/OpenCL when the original code is called through the new Python interface. Third, we verify that the final implementation is capable of running on the CUDA capable GPU backend and measure its performance dependency on the size of the lattice.

Experimental setup. The benchmarks are run in single precision mode and with 5000 simulation steps per reported measurement. The comparison of CPU/OpenCL is performed with the grid sizes of from 128×128 - 1024×1024 . The CUDA

GPU benchmark is run on NVIDIA Tesla V100S-PCIE-32GB GPU

available in the CINECA environment with sizes up to 2048×2048 . The performance is reported in Million Lattice Updates Per Second:

$$\text{MLUPS} = \frac{N_x N_y \cdot \text{steps}}{10^6 \cdot \text{time}}.$$

This metric is widely used to measure the performance of LBM implementations since it is directly correlated with number of updated lattice cells per second. Every reported point correspond to one complete benchmark with 5000 steps with the flag of final validation being true. The same input sizes were used for the comparison plot of two CPU/OpenCL

TABLE V
EXPERIMENTAL SETUP USED IN THE PROJECT.

Component	Configuration
Original baseline	miniLB C++/SYCL
Python wrapper	pybind11 + dpctl
Memory model	dpctl USM shared memory
CPU backend	OpenCL CPU
GPU backend	CUDA SYCL
GPU device	NVIDIA Tesla V100S-PCIE-32GB
Precision	Single precision
Iterations	5000
Metric	Million Lattice Updates/s (MLUPS)

TABLE VI
MAIN VALIDATION CHECKS USED BEFORE REPORTING PERFORMANCE.

Check	Purpose
Collision all populations	All D2Q9 populations are updated
Streaming populations	All D2Q9 streaming populations updated
Bounce-back walls	Wall bounce back behavior is present
Moving Lid	Lid-driven cavity condition is active
USM allocation	Arrays are allocated through dpctl USM
C++ kernel execution	The benchmark loop runs in native code
Final step check	Last reported state is valid

implementations. The GPU run used the same benchmark structure but a larger set of grid sizes, because the Tesla V100S can expose more parallelism on larger problems.

Reproducibility commands. The most relevant reproduction commands were included in experimental results. The CPU/OpenCL benchmark was conducted independently using standalone miniLB program, while V5 version was performed using Python and runtime device selector.

```
source /opt/intel/oneapi/setvars.sh

cmake -S . -B our_builds/build_single \
  -DCMAKE_BUILD_TYPE=Release \
  -DSYCL_IMPL=dpccpp \
  -DCMAKE_CXX_COMPILER=icpx \
  -DBGK_USE_CASE=LDC \
  -DBGK_PRECISION=SINGLE \
  -DBGK_SYCL_MALLOCSHARED=ON

ONEAPI_DEVICE_SELECTOR=opencl:cpu ./bgk2dSYCL

MINILB_DEVICE_SELECTOR="opencl:cpu:0" python3
  benchmark_v5_pybind11.py
MINILB_DEVICE_SELECTOR="cuda:gpu:0" python3
  benchmark_v5_pybind11_gpu_6cases.py
```

For the GPU test, the native extension was rebuilt for the CUDA SYCL target. The target and architecture flags were:

```
-fsycl-targets=nvptx64-nvidia-cuda
--cuda-gpu-arch=sm_70
```

Correctness checks. The performance numbers were reported only after passing the benchmark validation checks. They confirm collision and streaming over all D2Q9 populations, bounce-back wall handling, moving lid handling, Python-side USM allocation with dpctl, and C++ kernel execution through pybind11. It is important because otherwise the fast result would be meaningless since the implementation can change the physical update, e.g., by introducing only periodic boundaries.

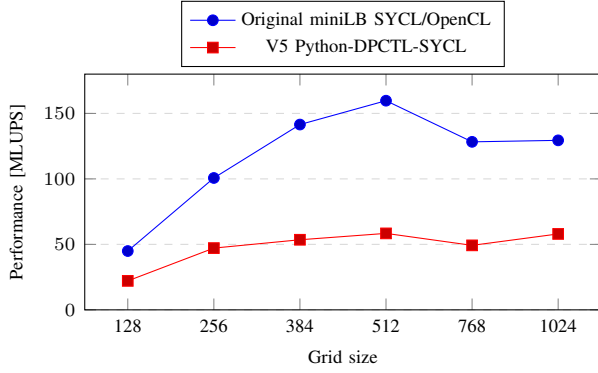


Fig. 3. CPU/OpenCL performance comparison between the original miniLB SYCL baseline and the Python-DPCTL-SYCL implementation.

TABLE VII
CPU/OPENCL COMPARISON BETWEEN ORIGINAL MINILB AND V5.

Grid	Original	V5	V5/original
128 ²	44.91	22.12	49.3%
256 ²	100.70	47.16	46.8%
384 ²	141.49	53.52	37.8%
512 ²	159.71	58.39	36.6%
768 ²	128.29	49.25	38.4%
1024 ²	129.43	57.97	44.8%

CPU/OpenCL comparison. Figure 3 the comparison of the original miniLB SYCL CPU/OpenCL baseline with the V5 Python-DPCTL-SYCL implementation. The original implementation is faster for all CPU sizes. This is expected since the original code is a specialized baseline, while V5 adds the Python-DPCTL path. However, the results of V5 are stable across all tested sizes and prove that the execution through Python interface works correctly

The V5 implementation is able to get up to 49.3% of the original performance in the 128×128 case and between 36.6% and 46.8% for the other tested CPU sizes. Thus, the main message from this experiment is not that the Python-DPCTL version outperforms the original CPU baseline, but that it allows preserving the correct C++/SYCL execution path together with the addition of Python queue control and USM interoperability.

It can be seen from the CPU results that the overhead is not constant and varies depending on the grid size. The ratio between the performance of the two implementations decreases for the smallest test case and some medium ones. That means that the final performance depends not only on the Python interface, but also on the memory access pattern and the kernel itself of the V5 implementation. Since both implementations execute compiled SYCL code, this comparison is useful for evaluating the overhead of the new Python-DPCTL execution path.

Another point from the CPU results is that the original CPU baseline performs its best at 512^2 , while the V5 path remains in a narrower range starting from 256^2 . This suggests that the Python-DPCTL version is not only affected by the amount of arithmetic work, but also by how the data path and the

TABLE VIII
V5 CUDA GPU RESULTS ON TESLA V100S.

Grid size	Time [s]	Performance [MLUPS]
128 ²	5.742	14.27
256 ²	6.219	52.69
384 ²	10.797	68.28
512 ²	15.877	82.55
768 ²	31.792	92.76
1024 ²	51.112	102.58
1280 ²	76.569	106.99
1536 ²	103.942	113.49
1792 ²	142.169	112.94
2048 ²	179.256	116.99

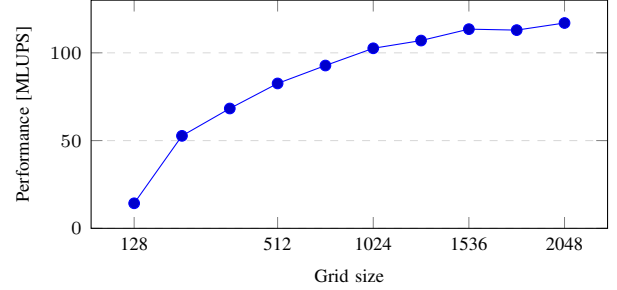


Fig. 4. V5 CUDA SYCL performance on the Tesla V100S as the grid size increases.

temporary arrays interact with the backend implementation. For this reason, the CPU results should be interpreted as an end-to-end measurement of the new interface rather than as an isolated measurement of one kernel.

CUDA GPU run. Finally, the V5 implementation was run on Tesla V100S using the CUDA SYCL backend. The performance grows with the grid size from 14.27 MLUPS 128×128 to 116.99 MLUPS at 2048×2048 . This behavior is reasonable because small grids suffer more from fixed overheads including queue management, kernel launch overhead and Python-to-C++ execution, while the largest test cases better represent the actual behavior of the GPU backend in case when the overhead costs are amortized by sufficiently large amount of computations.

Figure 4 shows the scaling trend of the V5 CUDA run. The curve grows rapidly from the smallest cases, and then becomes more stable for the largest grids. This confirms the interpretation that the small inputs suffer from the fixed overhead costs such as launch and orchestration overheads, while the larger inputs give more opportunities to execute parallel work on the GPU.

The largest GPU test cases are also valuable since they put the stress on the memory footprint and not just the launch overhead. The 2048^2 case passes the final validation check, which means that the Python-to-C++ path, USM allocations and CUDA SYCL execution work consistently for a significantly larger lattice than the CPU comparison range.

The table IX shows the comparison of two V5 backend paths on the sizes that were executed on both CPU/OpenCL and CUDA GPU. The smallest case is still faster on CPU/OpenCL, which is natural, since small workloads have higher overhead

TABLE IX
V5 PERFORMANCE ON CPU/OPENCL AND CUDA GPU FOR COMMON GRID SIZES.

Grid	CPU [MLUPS]	GPU [MLUPS]	GPU/CPU
128 ²	22.12	14.27	0.65×
256 ²	47.16	52.69	1.12×
384 ²	53.52	68.28	1.28×
512 ²	58.39	82.55	1.41×
768 ²	49.25	92.76	1.88×
1024 ²	57.97	102.58	1.77×

of the GPU. From 256², the CUDA backend starts to be faster for the V5 implementation, and this difference gets stronger for the larger sizes.

Original CUDA baseline issue. In order to compare against original miniLB on the CUDA GPU we tried to use the original minilb code as a CUDA GPU baseline. While it could be rebuild for the CUDA SYCL target, it resulted in an error at execution time :CUDA_ERROR_ILLEGAL_ADDRESS. For this reason, there was no valid original CUDA GPU MLUPS value that could be measured in the tested environment. The V5 implementation could be run on the same GPU and passed the correctness check, thus only GPU results of V5 path are reported. Keeping this failed run in the report is valuable, as it shows that the absence of the GPU comparison curve is caused by the failed run of the original CUDA baseline in the tested environment, and not by ignoring the baseline. The recorded error was:

```
CUDA_ERROR_ILLEGAL_ADDRESS
return_code = -6
MLUPS = None
```

Reproducibility notes. The logs of the benchmarks contain information about the selected backend, the name of the device, the execution time, the computed MLUPS value, and the validation flag. All this information was used to produce the tables and plots of this section. Moreover, the same number of simulation steps was used for all the tests, which makes the comparison easier.

Result interpretation. The recorded timings were kept together with the validation output. The reason for this is that the goal was not just to achieve the maximum MLUPS value, but to maintain the intended LBM behavior. In particular, the checks ensure collision, streaming, bounce-back walls, moving lid, Python-side USM allocation, and C++ kernel execution.

Discussion. The experiments have shown two aspects of the implementation. First, on the CPU/OpenCL backend the Python-DPCTL version is slower than the original miniLB baseline, but it maintains the same execution and adds Python control layer. Second, on the GPU backend the V5 implementation successfully runs on the Tesla V100S and reaches the best MLUPS value for the largest size. Also the failure of the original CUDA baseline shows the importance of documenting the exact behavior on the build and runtime when migrating SYCL code between backends.

The performance gap on CPU/OpenCL is not surprising. The V5 version introduces the more general Python-facing interface

and uses the queue and memory abstractions provided by dpctl. The advantage of such approach is not a faster CPU baseline, but the more flexible execution path where Python is able to prepare the experiment and rely on compiled kernels for the numerical computations. In this sense, the project serves as a test for interoperability and portability, and not an attempt at micro-optimizing the kernels.

Interpretation limitations. The benchmark comparison must be viewed as the evaluation of the whole implementation path, and not the micro-benchmark of individual kernels. As far as the GPU is concerned, the initial CUDA baseline did not offer the correct MLUPS measure, which is why the CUDA part is about running V5 and how scale is affected by the bigger lattices. **Lessons learned.** The primary conclusion to draw is that integrating a Python interface into an HPC program is not just about the issue of binding. It is also about queue ownership, USM memory, benchmark granularity, validation, and backend-specific compilation. Migrating the whole time loop to C++ was crucial, as it avoided the measurement of the repeated Python function calls instead of the LBM kernels.

V. CONCLUSIONS

This project implements the Python-DPCTL execution path for the miniLB Lattice Boltzmann code while maintaining the numerical computations in C++/SYCL. The final V5 version uses pybind11 for Python and C++ connection, dpctl for queue and device management, USM for data allocation, and the entire 5000 iterations loop implemented in compiled language.

The comparison between CPU/OpenCL backend results shows that the original implementation is still faster than the one in OpenCL, but V5 version correctly adds the required Python orchestration layer and confirm the computations of the LBM kernel for all sizes that were tested. The results of the CUDA implementation prove that the proposed design can also be applied to the real NVIDIA GPU backend and get 116.99 MLUPS on the 2048×2048 case on the Tesla V100S. Generally, the work demonstrates a viable approach of using Python orchestration and C++/SYCL LBM kernels through dpctl and USM memory. The main benefit of the proposed approach is that the user can continue using the Python orchestration part of the repeated numerical computations in native heterogeneous backend.

A natural next step would be tuning the GPU backend and profiling the reasons of the remaining CPU/OpenCL gap – the memory layout, queue behavior, or interoperability layer. Another possible improvement would be introducing more automated parameter sweeps from Python, since the current interface provides the means for this.

REFERENCES

- [1] S. Succi, *The Lattice Boltzmann Equation for Fluid Dynamics and Beyond*. Oxford University Press, 2001.
- [2] Khronos Group, *SYCL 2020 Specification*. Available: <https://registry.khronos.org/SYCL/>
- [3] IntelPython, *Data Parallel Control (dpctl) Documentation*. Available: <https://intelpython.github.io/dpctl/latest/>
- [4] W. Jakob, J. Rhineland, and D. Moldovan, *pybind11 documentation*. Available: <https://pybind11.readthedocs.io/>

- [5] University of Salerno HPC, *miniLB repository*. Available: <https://github.com/unisa-hpc/miniLB>
- [6] NVIDIA Corporation, *CUDA C++ Programming Guide*. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>